

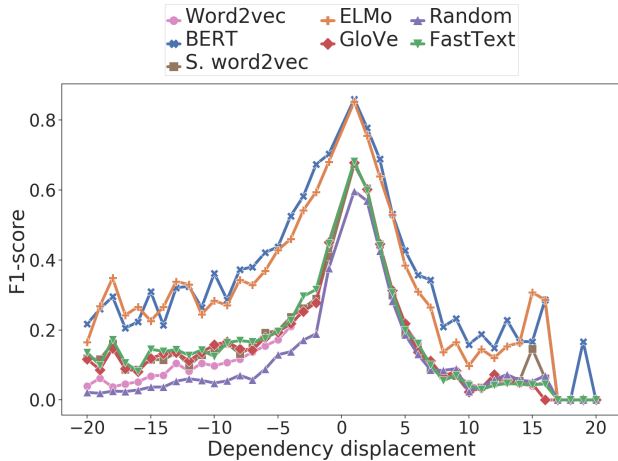


Figure 6: Dependency displacement F1-score on the EN-EWT UD test set for the $\mathcal{E}\mathcal{F}$ models (with gold segmentation)

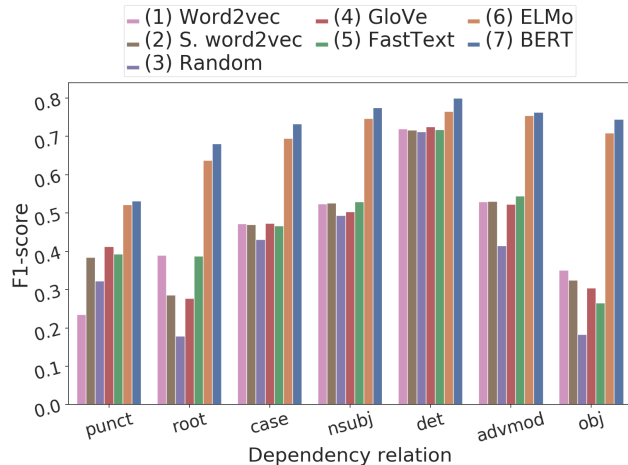


Figure 7: F1-score for the most common relations on the EN-EWT UD test set for $\mathcal{E}\mathcal{F}$ models (with gold segmentation)

ments. For contextualized vectors, the differences between both ELMo and BERT are small, and smaller than for constituents. They also perform closer to precomputed vectors, suggesting that these models could be less suited for dependency structures. The results also show similarities to the analysis on the constituent and dependency subtasks proposed by ? (?), who expose that ELMo and BERT perform closer to each other when they are asked to find out dependency relations. Figure 7 shows the performance on common dependency relations. Simple relations such as *det* are handled accurately by both precomputed and contextualized vectors,¹⁷ while harder ones such as *root*, *nsubj* or *obj* need deep contextualized ones.

¹⁷Other simple relations such as *punct* or *case* also showed a high performance, but only when evaluating relation labels only (i.e. when failing the head index was not penalized, as opposed to Figure 7). These results are not included due to space limitations.

In general, we feel Figures 4-7 distill that ELMo and BERT representations respond better to phrase-structure representations. Our intuition is that this might be due to language modelling objectives, where learning the concept of ‘phrase’ seems more natural than the one of ‘dependency’, although the answer to this is left as an open question.

Conclusion

We proposed a method to do constituent and dependency parsing relying solely on pretraining architectures – that is, without defining any parsing algorithm or task-specific decoders. Our goal was twofold: (i) to show to what extent it is possible to do parsing relying only in word vectors, and (ii) to study if certain linguistic structures are learned in pre-training networks. To do so, we first cast parsing as sequence labeling, to then map, through a linear layer, words into a sequence of labels that represent a tree. During training, we considered to both freeze and fine-tune the pretraining networks. The results showed that (frozen) pretraining architectures such as ELMo and BERT get a sense of the syntactic structures, and that a (tuned) BERT model suffices to parse. Also, by freezing the weights we have provided different analyses regarding the syntax-sensitivity of word vectors.

Contemporaneously to this work, ? (?) proposed to complement probing frameworks that test linguistic abilities of pretrained encoders with *control tasks*, i.e. tasks that can be only learned by the probing framework (e.g. classifying words into random categories). If the pretrained network is encoding the target property, the probing framework should perform well on the target task and poorly on the control one. As future work, we plan to add this strategy to our analyses, and expand our experiments to languages other than English.

Acknowledgements

We thank Mark Anderson and Daniel Hershcovich for their comments. DV, MS and CGR are funded by the ERC under the European Union’s Horizon 2020 research and innovation programme (FASTPARSE, grant No 714150), by the ANSWER-ASAP project (TIN2017-85160-C2-1-R) from MINECO, and by Xunta de Galicia (ED431B 2017/01). AS is funded by a Google Focused Research Award.

Voluptatum at aliquid incident alias quia magnam id ipsum ducimus, ipsam obcaecati doloremque ea at officia, nobis blanditiis nam necessitatibus iste suscipit delectus iusto eaque atque eveniet quod. Quis a expedita, nostrum iusto voluptate minus molestiae hic tempora non, vero nulla eveniet et sed aliquam optio, id illum perspiciatis iste beatae, quae aut delectus architecto accusantium quod aliquam eaque? Velit consectetur itaque et enim earum incident officii harum quos dignissimos, et est iure, similique sapiente aspernatur illo eaque, debitis ea quasi earum voluptatibus odit. Tenetur delectus quis ratione assumenda sapiente fugit ad et sunt, explicabo alias optio, tenetur natus nihil repellat est debitis maxime neque eius nobis distinctio minus. Animi illo doloribus cupiditate, totam neque consectetur accusantium recusandae deleniti, aliquam nisi illum. Accusamus inventore omnis aspernatur odio commodi laudantium officiis illum sequi, nostrum laudantium praesentium rem expedita

dignissimos incidunt animi autem similique?Nisi adipisci
unde dolores magni explicabo aliquid quam molestias ac-
cusamus, necessitatibus